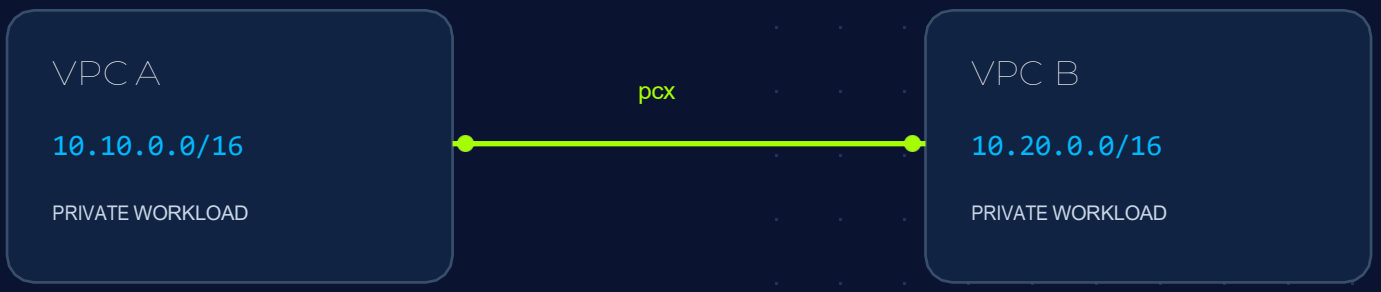
BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.

A guided networking project with EC2, custom route tables, security groups and a bastion host.



2 VPCs / 3 EC2 INSTANCES / 1 PRIVATE CONNECTION

AWS ARCHITECT ASSOCIATE

Name: **RAJAVIGNESH. SA** Batch: **1**

1. AIM

To establish a private network connection between two Amazon Virtual Private Clouds (VPCs) using VPC Peering and verify communication between two private EC2 instances using private IPv4 addresses.

2. OBJECTIVE

- To create two VPCs with non-overlapping CIDR blocks.
- To create public and private subnets.
- To configure route tables and subnet associations.
- To establish and activate a VPC Peering connection.
- To configure security groups for restricted access.
- To access private EC2 instances through a bastion host.
- To test connectivity before and after peering.
- To demonstrate a controlled failure and recovery.

3. PROJECT DESCRIPTION

EWB has an application in VPC A and another workload in VPC B. Both application servers must remain private and communicate using their private IPv4 addresses.

A public bastion host is created in VPC A for administration. VPC Peering connects VPC A and VPC B directly.

The project uses two VPCs, three EC2 instances and one private peering connection.

4. ARCHITECTURE

The project consists of the following components:

- **VPC A:** 10.10.0.0/16
- **VPC B:** 10.20.0.0/16
- **Public subnet A:** 10.10.1.0/24
- **Private subnet A:** 10.10.2.0/24
- **Private subnet B:** 10.20.1.0/24
- **Bastion host:** 10.10.1.10
- **App-A:** 10.10.2.10
- **App-B:** 10.20.1.10
- **Internet Gateway:** Attached to VPC A
- **VPC Peering:** Connects VPC A and VPC B

The bastion host provides administration access to the private instances. App-A and App-B communicate through the VPC Peering connection using their private IP addresses.

VPC B does not have an Internet Gateway and does not receive internet access through VPC A's Internet Gateway.

5. SOFTWARE AND SERVICES REQUIRED

- Amazon Web Services (AWS)
- Amazon VPC
- Amazon EC2
- Internet Gateway
- VPC Peering Connection
- Route Tables
- Security Groups
- Amazon Linux 2023
- EC2 Key Pair
- SSH client
- Terminal or WSL

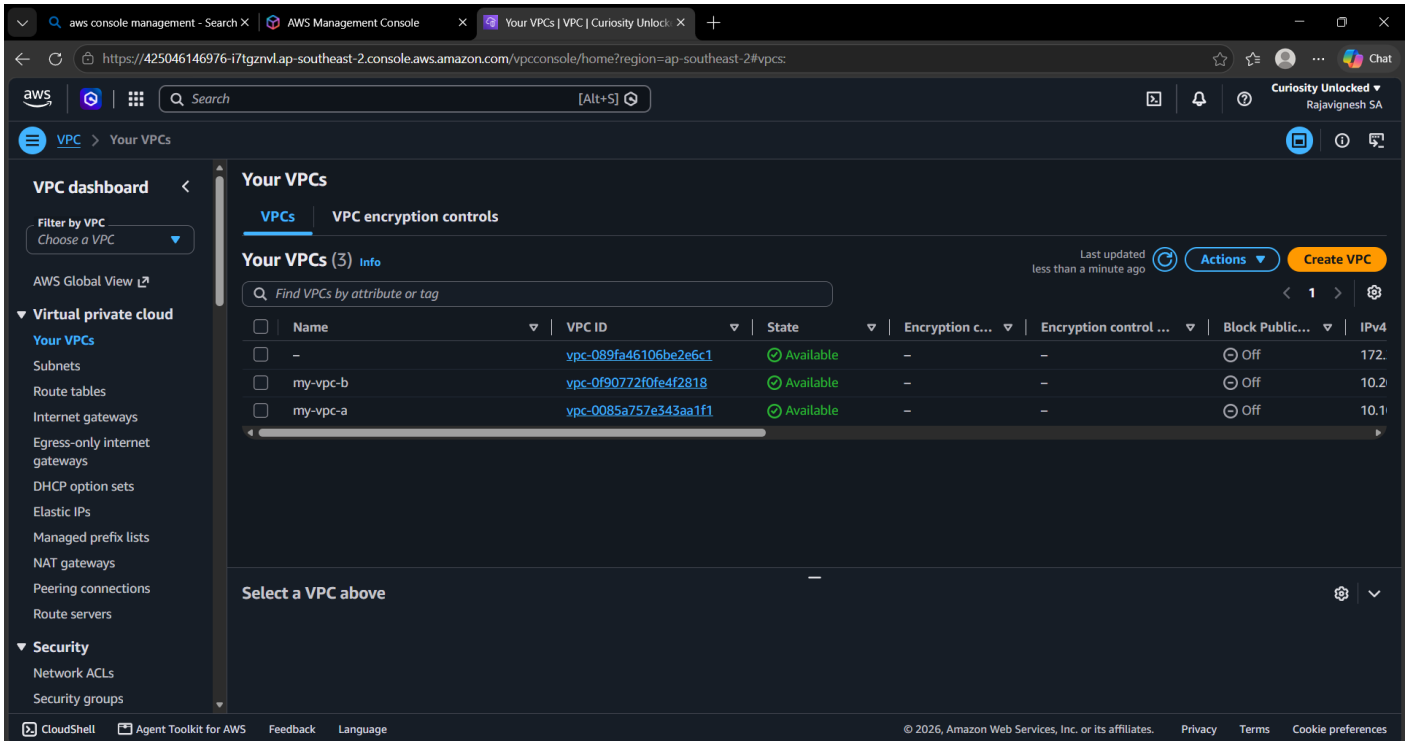
6. IP ADDRESS PLAN

Resource	Name	CIDR / IP Address
VPC A	my-vpc-a	10.10.0.0/16
Public subnet A	my-subnet-public	10.10.1.0/24
Private subnet A	My-subnet-private	10.10.2.0/24
VPC B	my-vpc-b	10.20.0.0/16
Private subnet B	my-vpc-private	10.20.1.0/24
Bastion host	my-bastion-1	10.10.1.10
Application server A	my-app-a	10.10.2.10
Application server B	my-app-b	10.20.1.10

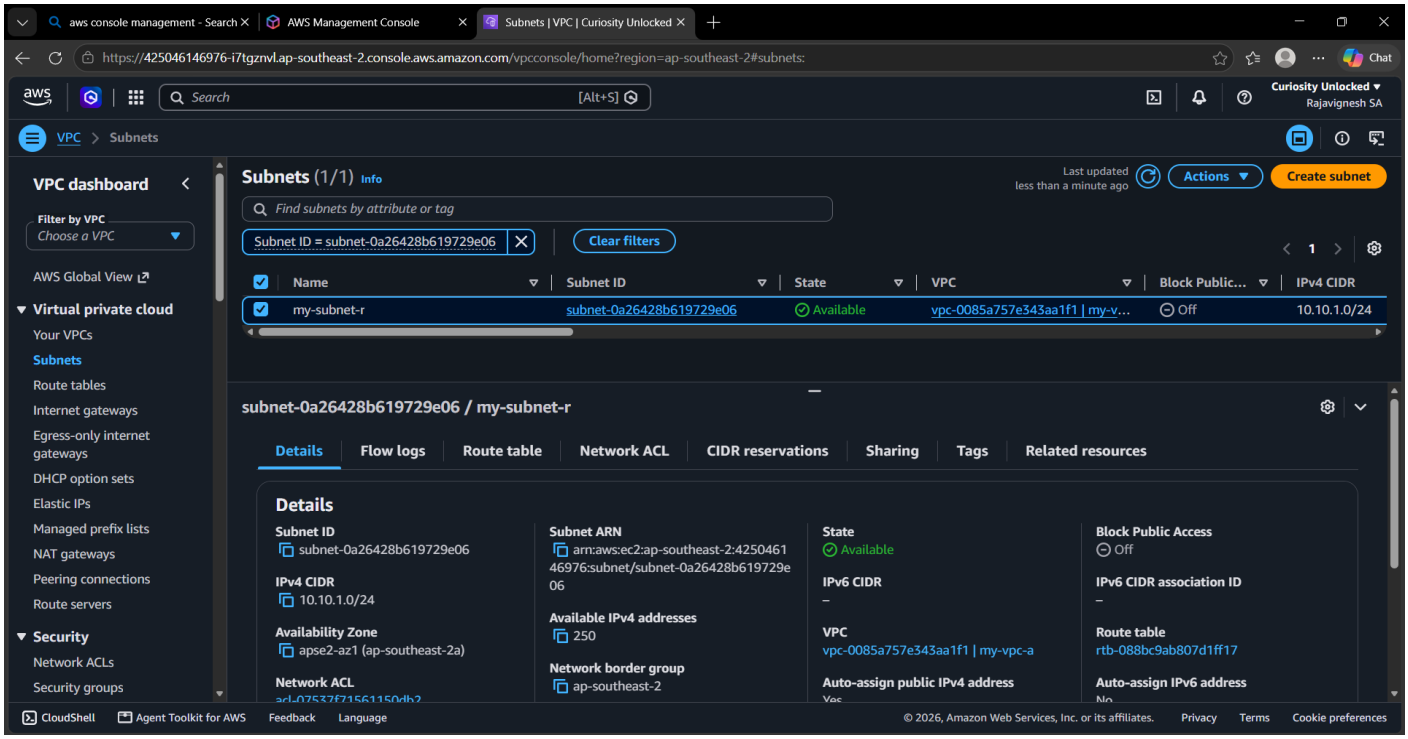
The two VPC CIDR blocks do not overlap, allowing them to be connected using VPC Peering.

7.SCREENSHOTS

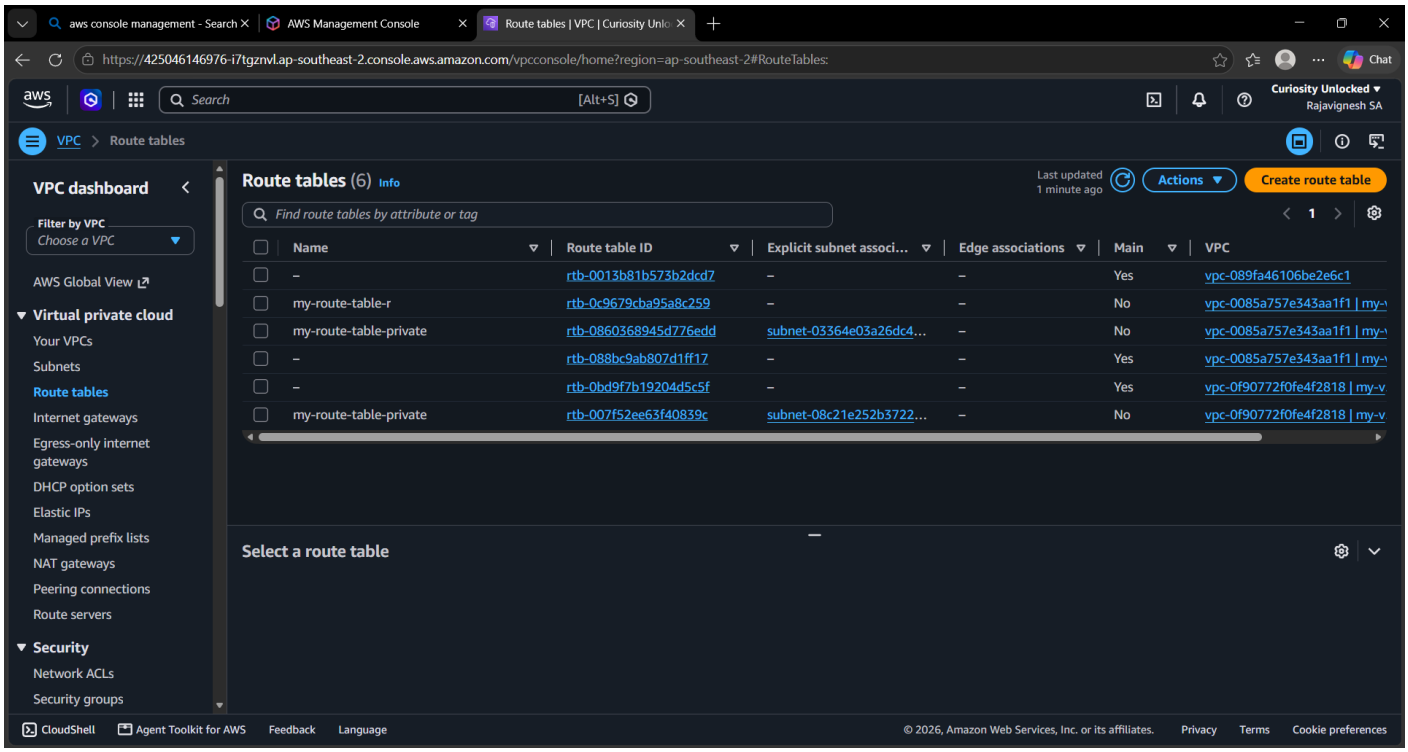
VPC's



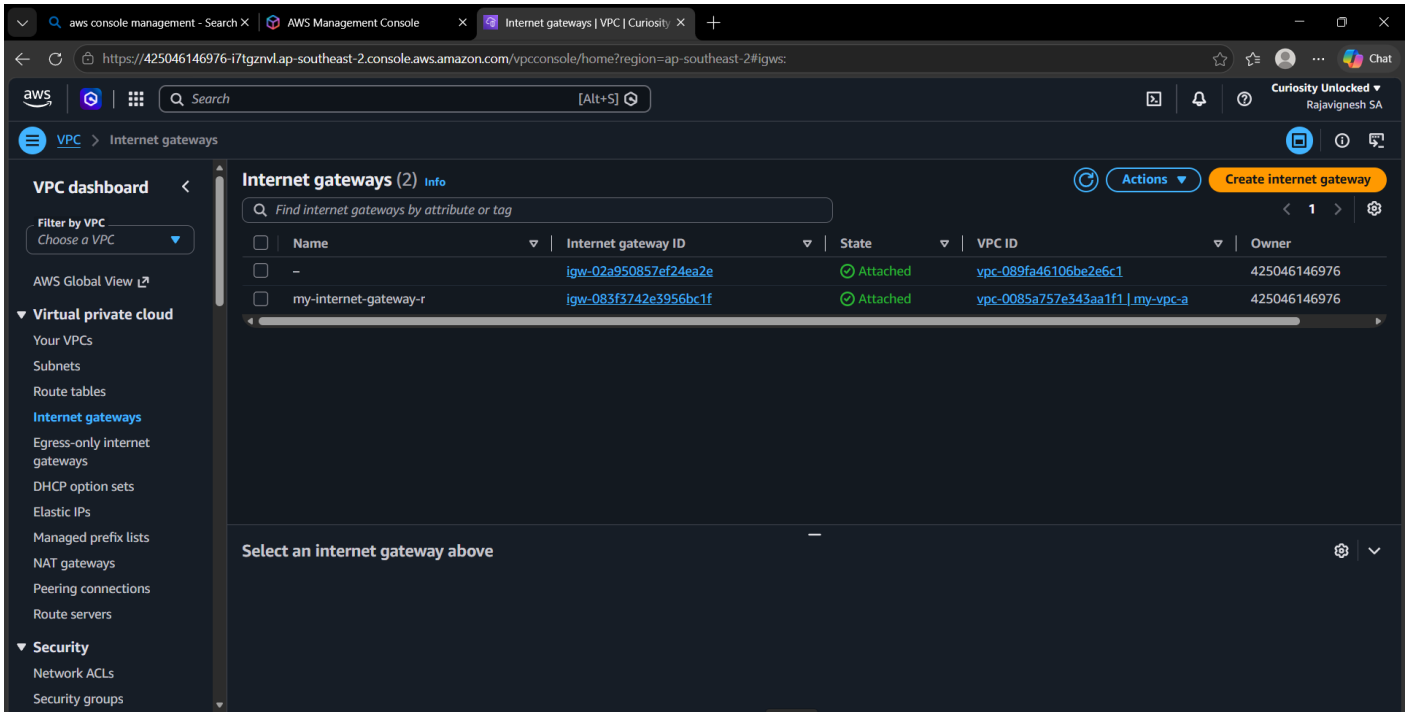
SUBNETS



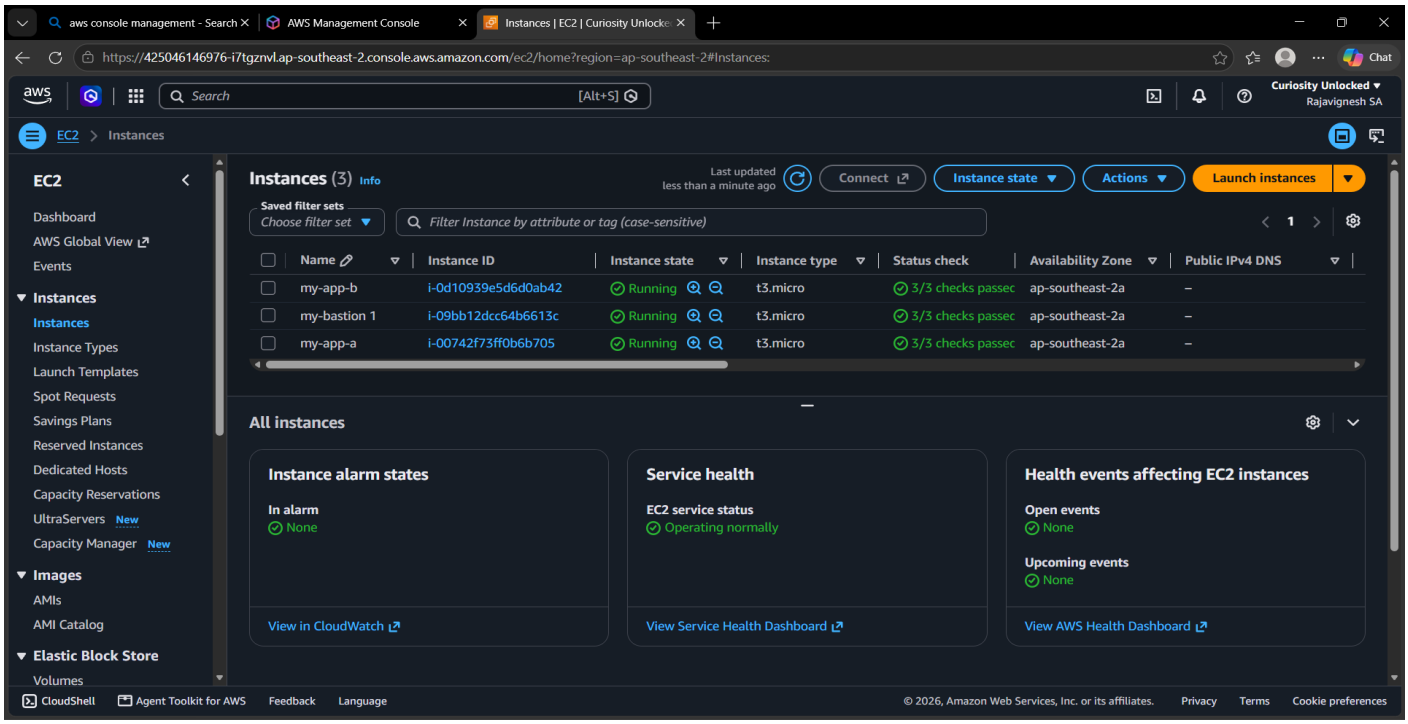
ROUTE TABLES



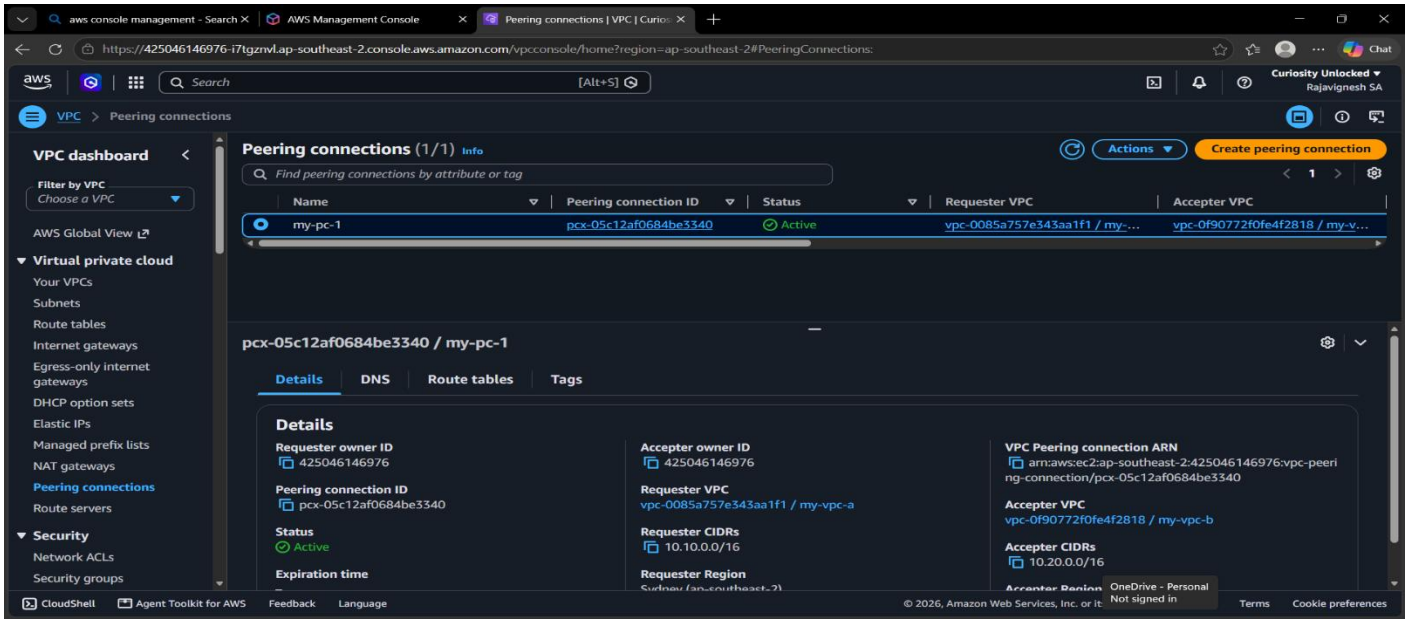
INTERNET GATEWAY



INSTANCES



PEERING CONNECTION



SSH connectivity

```
C:\Users\RAJAVIGNESH SA>ssh -i "%USERPROFILE%\Downloads\my-key-pair.pem" ec2-user@3.27.217.196
The authenticity of host '3.27.217.196 (3.27.217.196)' can't be established.
ED25519 key fingerprint is SHA256:H9gHdCingxO0qeReHhoQgirBZTp9Q+kj9OY5mvDy1rI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.27.217.196' (ED25519) to the list of known hosts.
       #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-1-125 ~]$
```

Bastion - App-A connectivity

```
[ec2-user@ip-10-10-1-125 ~]$  ping -c 4 10.10.2.60
PING 10.10.2.60 (10.10.2.60) 56(84) bytes of data.
64 bytes from 10.10.2.60: icmp_seq=1 ttl=127 time=0.171 ms
64 bytes from 10.10.2.60: icmp_seq=2 ttl=127 time=0.206 ms
64 bytes from 10.10.2.60: icmp_seq=3 ttl=127 time=0.200 ms
64 bytes from 10.10.2.60: icmp_seq=4 ttl=127 time=0.206 ms

--- 10.10.2.60 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3097ms
rtt min/avg/max/mdev = 0.171/0.195/0.206/0.014 ms
[ec2-user@ip-10-10-1-125 ~]$
```

Bastion - App-B connectivity

```
C:\Users\RAJAVIGNESH SA>ssh -i "%USERPROFILE%\Downloads\my-key-pair.pem" ec2-user@3.27.217.196
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Tue Oct 27 18:01:45 2026 from 27.60.165.111
[ec2-user@ip-10-10-1-125 ~]$ ping -c 4 10.20.210.77
PING 10.20.210.77 (10.20.210.77) 56(84) bytes of data.
64 bytes from 10.20.210.77: icmp_seq=1 ttl=127 time=0.213 ms
64 bytes from 10.20.210.77: icmp_seq=2 ttl=127 time=0.207 ms
64 bytes from 10.20.210.77: icmp_seq=3 ttl=127 time=0.182 ms
64 bytes from 10.20.210.77: icmp_seq=4 ttl=127 time=0.174 ms

--- 10.20.210.77 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3153ms
rtt min/avg/max/mdev = 0.174/0.194/0.213/0.016 ms
[ec2-user@ip-10-10-1-125 ~]$ ping -c 4 10.10.2.60
PING 10.10.2.60 (10.10.2.60) 56(84) bytes of data.

--- 10.10.2.60 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3118ms
```

Configure Security Groups

Create the following security groups and inbound rules.

Security Group	Type / Port	Source
my-bastion-sg	SSH / TCP 22	Your current public IPv4 address /32
my-app-a	SSH / TCP 22	10.10.1.10/32
my-app-a	ICMPv4 Echo Request	10.20.1.10/32
my-app-b	SSH / TCP 22	10.10.1.10/32
my-app-b	ICMPv4 Echo Request	10.10.2.10/32

Keep the default outbound allow-all rules for this lab. Do not expose SSH or ICMP to 0.0.0.0/0.

8. TROUBLESHOOTING

Bastion SSH times out:

- Check the current public IPv4 address of the bastion.
- Verify the SSH security-group source.
- Check the public subnet association.
- Verify the default route to the Internet Gateway.

App-A SSH fails:

- Verify App-A's private IP address.
- Check that SSH is allowed from 10.10.1.10/32.
- Verify the key pair and username.

App-B SSH fails:

- Check the public route table's peering route.
- Verify the return route in VPC B.
- Confirm that App-B allows SSH from the bastion's private IP.

Ping fails after peering becomes active:

- Check both private route tables.
- Verify the peering connection ID and Active status.
- Check the destination security-group ICMP rule.
- Confirm the actual private IP addresses.

Peering creation fails:

- Check all CIDR blocks associated with both VPCs for overlapping ranges.

Permission denied (publickey):

- Check the correct key pair, local IdentityFile and ec2-user username.

9. CLEANUP

After saving the screenshots and test evidence:

1. Terminate my-bastion-1, my-app-a and my-app-b.
2. Verify that unused lab EBS volumes and snapshots are removed.
3. Release any Elastic IP created for the project.
4. Delete the unused lab key pair if it is no longer required.
5. Remove the peer routes and delete the VPC Peering connection.
6. Delete the lab security groups after the instance network interfaces disappear.
7. Delete the lab subnets and custom route tables.
8. Detach and delete the Internet Gateway.
9. Delete both VPCs.
10. Check the AWS account for remaining project resources and possible charges.
11. Capture cleanup evidence.

10. CONCLUSION

The AWS VPC Peering project demonstrates how two non-overlapping VPCs can communicate using private IPv4 addresses. The project uses route tables, security groups and a bastion host to establish controlled private connectivity between application servers.

The successful completion of the project is verified through baseline testing, forward and reverse ping tests, private SSH access, controlled failure and recovery. Actual AWS screenshots and terminal outputs must be included as evidence.